

Thoughts on Primary Analysis for ADAPTS-HCT

November 23, 2025

In this document, we discuss the design of online agent for ADAPTS-HCT in terms of its validity for the primary after-study analysis. ADAPTS-HCT offers three types of interventions under different decision frequencies. Our algorithm is a three-agent algorithm, where each agent corresponds to one type of intervention. In this document, we focus on the discussion of one agent (AYA agent) that provides a binary action on a twice-daily basis. In Appendix ??, we discuss how to flatten the three-agent algorithm into a single-agent algorithm.

Our goal is to study a general framework for online agent design, where we allow pooling across participants for certain components in the online agent to accelerate learning, and allow adaptive sampling. (by allowing adaptive sampling, I mean we can do rigorous inference on the collected dataset.) ? proposed an all-purpose, general, statistical inference approach for adaptive sampling for longitudinal data with pooling across participants. However, *this all-purpose approach imposes assumptions on the behavior policies induced by the online agent*. Notable assumptions that may limit the applicability of their approach include

- **parametric form:** the behavior policy π_t must be parametrized by a finite-dimensional estimator $\hat{\beta}_t^{(n)}$, which excludes many non-parametric behavior policies (aka, the dimension of $\hat{\beta}_t^{(n)}$ should not be large compared to n);
- **consistency:** there exists unknown parameters, $\beta_t^{(\infty)}$ for which $\hat{\beta}_t^{(n)}$ must be a consistent estimator. That is the estimators, $\hat{\beta}_t^{(n)} \xrightarrow{p} \beta_t^{(\infty)}$, as $n \rightarrow \infty$ for all t ;
- **differentiability:** the behavior policy π_t and the estimating function ϕ_t must be differentiable w.r.t. the parameters.

In this document, we propose a **two-layer online agent design** that allows us to apply the general purpose inference approach developed in ?, while allowing for a flexible online agent design. In short, the two-layer online agent design involves a learning algorithm and an optimization algorithm:

- **the learning algorithm:** learns on each participant’s own data and outputs estimators for the participant’s underlying parameters;
- **the optimization algorithm:** views the person-specific estimators as person-specific features. The algorithm inputs these features at each time t along with each participants’ state at time t , and outputs the sampling probabilities for each participant.

We argue that the learning algorithm can be *flexible*: it may involve non-smooth/non-parametric operations on the observed data. The learned features need not be consistent estimators of person-specific parameters(they are likely only consistent under assumptions that each person is an MDP). The optimization algorithm, on the other hand, will satisfy the assumptions by the general-purpose inference approach developed in ?.

Notation list

- $s_{i,t}, a_{i,t}, r_{i,t}$: the state, action and reward at step t of episode i
- $\pi_{i,t}$ the behavior policy at step t of episode i
- θ_i : some true underlying parameter about episode i
- $\hat{\theta}_{i,t}$: the prediction of the true underlying parameter from the learning algorithm at step t of episode i
- $\hat{\beta}_{i,t}$: the parameter the optimization outputs used as inputs for the behavior policy

1 Formulation

We formulate the ADAPTS-HCT as an RL problem with a sequence of episodes $i = 1, \dots, N$. Each episode i starts randomly at time t_i . We may think of each episode as a participant (dyad) in our clinical trial, and t_i is the random variable indicating the time when the i -th participant is enrolled.

Each episode i consists of a sequence of transitions $(\mathbf{s}_{i,t}, a_{i,t}, r_{i,t}, \pi_{i,t})_{t=t_i}^{t_i+T_i-1}$, where $\mathbf{s}_{i,t}$ is the state, $a_{i,t}$ is the action, $\pi_{i,t}$ is the sampling probability, and $r_{i,t}$ is the reward (medication adherence) at time t of episode i . In ADAPTS-HCT, each dyad stays for 14 weeks ($T_i = 14 \times 14 = 196$ for all i). Here the state space is all the observed variables at the time t that may include *AYA's past medication adherence, relational quality, psychological distress, and positive affect* etc.

In ADAPTS-HCT, we have a binary action space $\mathcal{A} = \{0, 1\}$ for the AYA agent representing whether a positive psychology message is sent to the AYA at time t , and the sampling probability $\pi_{i,t} \in [0, 1]$ is the probability of taking action $a_{i,t} = 1$.

We denote the trajectory of episode i at the time t as $\mathcal{H}_t^{(i)} = \{(\mathbf{s}_{i,t'}, a_{i,t'}, r_{i,t'}, \pi_{i,t'})\}_{t'=t_i}^{\min\{t-1, t_i+T_i-1\}} \cup \{\mathbf{s}_{i,t}\}$.

2 Two-layer online agent design

Our online agent consists of a sequence of learning algorithms and one optimization algorithm, denoted by $\mathcal{L}_i$ and $\mathcal{O}$ respectively. Here $\mathcal{L}_i$ is the learning algorithm for episode i , which is a mapping from the episode i 's own trajectory $\tau_{i,t}$ to a prediction $\hat{\theta}_{i,t}$ about the underlying parameters in which episode i runs. This prediction could represent some marginal statistics about the underlying environments that inform decision making, e.g., each arm's mean reward. At time t , the optimization algorithm $\mathcal{O}$ takes input the $\{\hat{\theta}_{i,t}\}_{t_i \leq t, i \in [N]}$ from all the existing episodes and outputs the sampling probabilities for every ongoing episode.

3 Examples

This framework subsumes many existing online learning algorithms.

3.1 Thompson Sampling with Full Pooling

To recover TS with full pooling, set $\hat{\theta}_{i,t} = (\hat{\theta}_{i,a,t})_{a \in \mathcal{A}}$, where $\hat{\theta}_{i,a,t} = (\mathbf{B}_{i,a,t}, \mathbf{b}_{i,a,t})$, and

$$\mathbf{B}_{i,a,t} = \sum_{\tau=t_i}^{t-1} \mathbb{I}\{a_{i,\tau} = a\} \mathbf{s}_{i,\tau} \mathbf{s}_{i,\tau}^\top, \quad \mathbf{b}_{i,a,t} = \sum_{\tau=t_i}^{t-1} \mathbb{I}\{a_{i,\tau} = a\} \mathbf{s}_{i,\tau} r_{i,\tau}.$$

Then we choose the optimization algorithm $\mathcal{O}$ such that the behavior policy $\pi_{i,t}(a | \mathbf{s}) = \mathbb{E}[\text{softmax}(\mathbf{Z}\mathbf{s})]$ for all $a \in \mathcal{A}$. Here $\mathbf{Z} = (\mathbf{Z}_a)_{a \in \mathcal{A}} \in \mathbb{R}^{|\mathcal{A}| \times d}$ and $\mathbf{Z}_a \sim \mathcal{N}(\hat{\boldsymbol{\mu}}_{a,t}, \hat{\boldsymbol{\Sigma}}_{a,t-1})$, with

$$\hat{\boldsymbol{\mu}}_{a,t} = \left(\sum_i \mathbf{B}_{i,a,t} + \lambda \mathbf{I} \right)^{-1} \left(\sum_i \mathbf{B}_{i,a,t} \mathbf{b}_{i,a,t} \right), \quad \hat{\boldsymbol{\Sigma}}_{a,t} = \left(\sum_i \mathbf{B}_{i,a,t} + \lambda \mathbf{I} \right)^{-1}.$$

3.2 Pooling on the Parameter Space

We describe a potential algorithm design framework that fits into this framework. [ZP:Not sure about the name yet.] [SAM: connect to empirical Bayes, "EB-MTRL" or in line with very classical statistics: "Two-Stage RL"—see raudenbush paper in dropbox folder for this language...] [ZP:I like EB-MTRL. Two-Stage might be too general.] [SAM: related work will include horizontal federated RL (HFRL). The big difference between what we aim to do here and what is done in FL, is that the federated server sends the federated model weights back to RL agents and the RL agents use these federated model weights to update their local model. Instead our RL agents only use the federated



Figure 1: Online agent design with learning and optimization algorithms. [ZP:Add $s_{i,t}$ to this plot]

model weights to select actions, not to update their local model.] [ZP:I am against using the word federated RL. It is true that we are doing similar things. But it might confuse people but the motivation here. Also federated RL is a big literature.]

Assume that the environment for each episode i can be described by a random variable $\mathcal{E}_i$, sampled i.i.d. in the beginning of the episode from the prior distribution $\mathcal{E}_i \sim \mathcal{P}$. This is a simple pooling on the parameter space.

We go through this framework in its simplest form—equal-weighted pooling.

Learning component. [SAM: I suspect that we are pretending that the learning algorithm knows whether the underlying true environment is an MDP or contextual bandit and is using this knowledge. Do we want to pretend this for this paper?] [ZP:So far all the discussions is about algorithm. The environment can be anything.]

The goal of the learning component is to learn the target parameter θ_i^* of episode i that is a function of the underlying environment $\mathcal{E}_i$, and θ_i^* should concern the desired policy of episode i . For example, if the underlying environment for episode i is a stationary contextual bandit problem, θ_i^* is the coefficient for the true reward generating process. In case of RL, θ_i^* is the parameter of the optimal Q value function of episode i .

Specifically, for each episode i , we are interested in solving the following estimating equation:

$$\mathbf{0} = \mathbb{E}_{\mathcal{E}_i}[\mathbf{g}_t(\mathbf{s}, a, r; \theta_i^*)],$$

for some random function $\mathbf{g}_t$ measurable w.r.t the sigma-algebra generated by the trajectory up to step t of episode

i. One may think of the following example for a stationary contextual bandit algorithm:

$$\mathbf{g}_t(\boldsymbol{\theta}) = \begin{pmatrix} \sum_{t'=1}^t \frac{1_{a_{i,t'}=1}}{\pi_{i,t'}(1|\mathbf{s}_{i,t'})} \mathbf{s}_{i,t'} (r_{i,t'} - \boldsymbol{\theta}_1^\top \mathbf{s}_{i,t'}) \\ \dots \\ \sum_{t'=1}^t \frac{1_{a_{i,t'}=|\mathcal{A}|}}{\pi_{i,t'}(|\mathcal{A}||\mathbf{s}_{i,t'}|)} \mathbf{s}_{i,t'} (r_{i,t'} - \boldsymbol{\theta}_{|\mathcal{A}|}^\top \mathbf{s}_{i,t'}) \end{pmatrix}$$

We may also do long-term planning (i.e., RL) in this framework. For example, we can model the value function under a linear MDP model with feature mapping ϕ , the estimating equation becomes

$$\mathbf{g}_t(\boldsymbol{\theta}) = \left(\sum_{t'} \frac{1_{a_{i,t'}=a}}{\pi_{i,t'}(a|\mathbf{s}_{i,t'})} \phi(\mathbf{s}_{i,t'}, a) \left(r_{i,t'} + \gamma \arg \max_{a' \in \mathcal{A}} \langle \phi(\mathbf{s}_{i,t'}, a'), \boldsymbol{\theta}_{a'} \rangle - \boldsymbol{\theta}_a^\top \phi(\mathbf{s}_{i,t'}, a) \right) \right)_{a \in \mathcal{A}}.$$

Here one may think of $\boldsymbol{\theta}_i^*$ as the parameter of the optimal Q value function of episode i . [ZP:Note that here we can do hard max in the learning component. Might be an advantage.] [SAM: if, by chance, each participant(AKA, environment) is an MDP and T_i is large then I suspect the overall, average across-participants-regret will be improved? I don't have a good conjecture but here is a not-so-good conjecture: if we were in this setting then for each $i = 1, \dots, N$ at time t we would have a $Q(s, a; \mathcal{H}_t^{(i)}) = Q(s, a; \boldsymbol{\theta}_{i,t})$ so we could send these functions to optimization alg. This would reduce dimensionality if we don't include trait data.] [ZP:I don't quite follow here. When you say, "improved", are you saying improved over individual learning where there is no pooling in any sense?]

Optimization component. The optimization component here can be further decomposed into two layers: the first layer is the aggregation of the evidence from all the episodes, and the second layer is the optimization over the behavior policy.

The first layer: aggregation of the evidence. It tries to improve the posterior prediction of the target parameter $\boldsymbol{\theta}_i^*$ of episode i by taking into account the predictions from all the existing episodes. It can be seen as an aggregation of the evidence from all the episodes. For this purpose, we write the behavior policy parameter $\hat{\boldsymbol{\beta}}_t = (\hat{\boldsymbol{\beta}}_{i,t})_{i=1}^N$. Note I have a $\hat{\boldsymbol{\beta}}_{i,t}$ for each episode i to allow different aggregation for different episodes, in case we need to do a weighted average with varying weights.

A simple aggregation scheme is to take the average of predictions from all the existing episodes:

$$\hat{\boldsymbol{\beta}}_{i,t} = \frac{1}{\sum_{j=1}^N 1_{t_j \leq t}} \sum_{j=1}^N \hat{\boldsymbol{\theta}}_{j,t} 1_{t_j \leq t}.$$

[ZP: A potential generalization is the average on the function space. Think of $\hat{\boldsymbol{\theta}}_{i,t}(s, a)$ itself being a value function. Then the average over the function space is

$$\bar{\boldsymbol{\beta}}_{i,t}(s, a) = \frac{1}{\sum_{j=1}^N 1_{t_j \leq t}} \sum_{j=1}^N Q(s, a; \hat{\boldsymbol{\theta}}_{j,t}) 1_{t_j \leq t}.$$

If we can generalize Kelly's math to include behavior policy conditioned on functions instead of parameters, we can make this work. We need to use empirical process over function class indexed by s, a with the right covering number argument.]

The estimation equation for the behavior policy parameter is simply given by

$$\mathbf{0} = \mathbb{E} \left[\sum_{j=1}^N 1_{t_j \leq t} \left(\hat{\boldsymbol{\theta}}_{j,t} - \boldsymbol{\beta}_{i,t} \right) \right].$$

[ZP: With our generalization, for each s, a

$$\mathbf{0} = \sum_{j=1}^N \left(Q(s, a; \hat{\boldsymbol{\theta}}_{j,t}) - \hat{\boldsymbol{\beta}}_t(s, a) \right) 1_{t_j \leq t}$$

This would work very easily if each user-specific Q is assumed to be linear in the $\boldsymbol{\theta}_{i,t}$'s, i.e. $Q_i(s, a) := \phi(s, a)^T \boldsymbol{\theta}_{i,t}$ for state feature mapping, $\phi(s, a)$. Then estimating eqn is

$$\mathbf{0} = \sum_{j=1}^N 1_{t_j \leq t} \phi(s, a)^T \left(\hat{\boldsymbol{\theta}}_{j,t} - \boldsymbol{\beta}_{i,t} \right)$$

and you get the softmax below.]

Note that here it is easy to differentiate the objective function w.r.t. β_t .

[SAM: But non-linear Q ? We should be able to figure this out... Suppose we used a 2 layer neural net to model Q . At each time t for each individual i we would have a large p dim vector of estimated weights, $x_{i,t}$. If $p \ll N$ then we could just act as if the user-specific RL was only used to do feature construction resulting in p features. We could play the empirical bayes game as long as we solve the var-cov issue. However if p is not much less than N then we need to be more thoughtful...]

[ZP: If we are interested in deep model/non-linear model, would it be possible for us to consider Bayesian Neural Network?] [SAM: not sure, but worth investigating...]

The second layer: optimization over the behavior policy. It optimizes over the behavior policy using the aggregated evidence $\hat{\beta}_{i,t}$. Here to ensure the differentiability of the behavior policy w.r.t. β , and a Lipschitz condition on the behavior policy w.r.t. β , we use the following form of the behavior policy:

$$\pi_t(a \mid s; \hat{\beta}_{i,t}) = \text{softmax} \left(\hat{\beta}_{i,t}^\top \phi(s, \cdot) \right)_a,$$

where ϕ is the feature mapping. [SAM: so empirical Bayes produces a "more intelligent" policy, aka, one which incorporates uncertainty...] [ZP: Yes]

4 Empirical Bayes: A Contextual Bandit Example

We consider a principled framework here. Let us assume that the true underlying environment of episode i is contextual bandit with true coefficients $\theta_{i,a}$:

$$r_{i,t} = \theta_{i,a_{i,t}}^\top \mathbf{s}_{i,t} + \epsilon_{i,t}, \quad (1)$$

where $\epsilon_{i,t} \sim \mathcal{N}(0, \sigma^2)$ is i.i.d Gaussian noise. We further assume that $\{\mathbf{s}_{i,t}\}_{t \geq 1, i \geq 1}$ is also i.i.d.

We assume that $\theta_{i,a} \sim \mathcal{N}(\mu_a, \Sigma_a)$ i.i.d for each $a \in \mathcal{A}$ and $i \geq 1$, where μ_a is the prior mean and Σ_a is the prior variance. The idea of empirical Bayes is that we may use the data from the other episodes to learn about (μ_a, Σ_a) .

Specifically, the **learning algorithm** is

$$\hat{\mu}_{i,a,t} = \left(\frac{1}{\sigma^2} \sum_{\tau=1}^{t-1} 1_{\{a_{i,\tau}=a\}} \mathbf{s}_{i,\tau} \mathbf{s}_{i,\tau}^\top + \lambda \mathbf{I} \right)^{-1} \left(\frac{1}{\sigma^2} \sum_{\tau=1}^{t-1} 1_{\{a_{i,\tau}=a\}} \mathbf{s}_{i,\tau} r_{i,\tau} \right), \quad (2)$$

and

$$\hat{\Sigma}_{i,a,t} = \left(\frac{1}{\sigma^2} \sum_{\tau=1}^{t-1} 1_{\{a_{i,\tau}=a\}} \mathbf{s}_{i,\tau} \mathbf{s}_{i,\tau}^\top + \lambda \mathbf{I} \right)^{-1} \quad (3)$$

Then the learning algorithm's prediction is $\hat{\theta}_{i,t} = (\hat{\mu}_{i,a,t}, \hat{\Sigma}_{i,a,t})_{a \in \mathcal{A}}$.

The optimization algorithm first aggregates the information from each trajectory to estimate the true μ_a and Σ_a . The prediction is given by

$$\hat{\mu}_{a,t} = \left(\sum_{i=1}^N \hat{\Sigma}_{i,a,t}^{-1} \right)^{-1} \sum_{i=1}^N \hat{\Sigma}_{i,a,t}^{-1} \hat{\mu}_{i,a,t}, \quad (4)$$

and

$$\hat{\Sigma}_{a,t} \leftarrow \frac{1}{N} \sum_{i=1}^N (\hat{\mu}_{a,t} - \hat{\mu}_{i,a,t})^{\otimes 2} - \frac{1}{N} \sum_{i=1}^N \hat{\Sigma}_{i,a,t} \quad (5)$$

We further run Bayesian regression on episode i 's own data to get the posterior of $\theta_{i,a}$ given our predicted prior $(\hat{\mu}_{a,t}, \hat{\Sigma}_{a,t})$ and obtain:

$$\hat{\theta}_{i,a,t}^{\text{post}} = \left(\bar{\Sigma}_{i,t}^{-1} + \hat{\Sigma}_{a,t}^{-1} \right)^{-1} \left(\bar{\Sigma}_{i,t}^{-1} \hat{\mu}_{i,a,t} + \hat{\Sigma}_{a,t}^{-1} \hat{\mu}_{a,t} \right), \quad \hat{\Sigma}_{i,a,t}^{\text{post}} = \left(\bar{\Sigma}_{i,t}^{-1} + \hat{\Sigma}_{a,t}^{-1} \right)^{-1} \quad (6)$$

where

$$\bar{\Sigma}_{i,t}^{-1} = \frac{1}{\sigma^2} \sum_{\tau=1}^{t-1} 1_{\{a_{i,\tau}=a\}} \mathbf{s}_{i,\tau} \mathbf{s}_{i,\tau}^\top \quad (7)$$

These will be the parameter $\hat{\beta}_{i,t} = \left(\hat{\theta}_{i,a,t}^{\text{post}}, \hat{\Sigma}_{i,a,t}^{\text{post}} \right)_a$ for the behavior policy for episode i . From here we can use Thompson sampling with the posterior is Gaussian with parameters $\hat{\beta}_{i,t}$ and a smooth allocation function. Specifically, sample

$$\tilde{\theta}_{i,a,t} \sim \mathcal{N}(\hat{\theta}_{i,a,t}^{\text{post}}, \hat{\Sigma}_{i,a,t}^{\text{post}}) \quad (8)$$

and sample action $a_{i,t}$ from $\text{softmax}((\tilde{\theta}_{i,a,t}^\top \mathbf{s}_{i,t}))$. The behavior policy is given by

$$\pi_{i,t}(a \mid \mathbf{s}_{i,t}) = \Pr(a_{i,t} = a \mid \hat{\beta}_{i,t}, \mathbf{s}_{i,t}). \quad (9)$$

4.1 Some Justification

Proposition 1 (Consistency). *Under Regularity Assumptions, for any fixed t , $\hat{\mu}_{a,t} \rightarrow \mu_a$ and $\hat{\Sigma}_{a,t} \rightarrow \Sigma_a$ as $N \rightarrow \infty$.*

Proof. For simplicity, let us assume that there is no overlap in episodes. That is episode $i + 1$ always starts after episode i has ended.

We have that

$$\hat{\mu}_{a,t} = \left(\sum_{i=1}^N \hat{\Sigma}_{i,a,t}^{-1} \right)^{-1} \sum_{i=1}^N \hat{\Sigma}_{i,a,t}^{-1} \hat{\mu}_{i,a,t} \quad (10)$$

$$= \left(\sum_{i=1}^N \hat{\Sigma}_{i,a,t}^{-1} \right)^{-1} \sum_{i=1}^N \hat{\Sigma}_{i,a,t}^{-1} (\theta_{i,a} + \Sigma_{i,a,t} / \sigma^2) \quad (11)$$

$$(12)$$

□

We further need some results on the optimality results. Is the above estimator of the prior optimal?

5 Estimating Equation for Primary Analysis in ADAPTS-HCT

5.1 Medication adherence.

[SAM: Would prefer one estimating equation for all.. need to think on this—should be a causal excursion effect. This is VERY interesting! Another approach is to prespecify the primary analysis to only concern AYA intervention but include other intervention indicators as part of state in this analysis... I think this is what you are doing below in AYA intervention..] We are interested in the treatment effects of three interventions onto medication adherence denoted by ADH_t . We first consider the individual effect of each intervention.

Here we use feature construction $\phi^{\text{AYA}}(S_t)$, $\phi^{\text{CARE}}(S_d)$, and $\phi^{\text{GAME}}(S_w)$ to denote the features mapping for the three interventions respectively. These features are included to reduce the noise in estimating treatment effects. Here S_t is the all the observed state variables up to time t .

AYA intervention onto medication adherence. [SAM: discuss collinearity issue...]

$$\mathbb{E} \left[\left(\text{ADH}_t - \phi^{\text{AYA}}(S_t)^\top \theta^{\text{AYA}} - \pi_t^{\text{AYA}} - (A_t^{\text{AYA}} - \pi_t^{\text{AYA}}) \tau^{\text{AYA}} \right) \begin{bmatrix} \phi^{\text{AYA}}(S_t) \\ \pi_t^{\text{AYA}} \\ A_t^{\text{AYA}} - \pi_t^{\text{AYA}} \end{bmatrix} \right] = 0.$$

Care-partner intervention onto medication adherence. This only happens at the morning decision time in a day. We use day index d to denote the day. Recall that $t(d)$ is the time index of the morning decision time in day d .

$$\mathbb{E} \left[\left(\text{ADH}_{t(d)} + \text{ADH}_{t(d)+1} - \phi^{\text{CARE}} S_d^\top \theta^{\text{CARE}} - \pi_d^{\text{CARE}} - (A_d^{\text{CARE}} - \pi_d^{\text{CARE}}) \tau^{\text{CARE}} \right) \begin{bmatrix} \phi^{\text{CARE}}(S_d) \\ \pi_d^{\text{CARE}} \\ A_d^{\text{CARE}} - \pi_d^{\text{CARE}} \end{bmatrix} \right] = 0.$$

This expectation is taken for $A_{t(d)}^{\text{AYA}}$ and $A_{t(d)+1}^{\text{AYA}}$ chosen by some converged policy $\pi^{\text{AYA}} = (\pi_1^{\text{AYA}}, \dots, \pi_T^{\text{AYA}})$ for $T = 14 \times 14$.

Game intervention onto medication adherence. This only happens at the Sunday evening in a week. We use week index w to denote the following week. Recall that $t(w)$ is the time index of the Monday morning decision time in week w .

$$\mathbb{E} \left[\left(\sum_{t=t(w)}^{t(w)+13} \text{ADH}_t - \phi^{\text{GAME}} S_w^\top \theta^{\text{GAME}} - \pi_w^{\text{GAME}} - (A_w^{\text{GAME}} - \pi_w^{\text{GAME}}) \tau^{\text{GAME}} \right) \begin{bmatrix} \phi^{\text{GAME}}(S_w) \\ \pi_w^{\text{GAME}} \\ A_w^{\text{GAME}} - \pi_w^{\text{GAME}} \end{bmatrix} \right] = 0.$$

5.2 Psychological distress and positive affect.

We denote by Stress_d and Positive_d the psychological distress and positive affect measured at the evening decision time in day d .

AYA intervention onto AYA psychological distress and positive affect.

$$\mathbb{E} \left[\left(\text{Stress}_d^{\text{AYA}} - \phi^{\text{AYA}}(S_{t(d)})^\top \theta^{\text{AYA}} - \pi_{t(d)}^{\text{AYA}} - (A_{t(d)}^{\text{AYA}} - \pi_{t(d)}^{\text{AYA}}) \tau^{\text{AYA}} \right) \begin{bmatrix} \phi^{\text{AYA}}(S_{t(d)}) \\ \pi_{t(d)}^{\text{AYA}} \\ A_{t(d)}^{\text{AYA}} - \pi_{t(d)}^{\text{AYA}} \end{bmatrix} \right] = 0.$$

Care-partner intervention onto care-partner psychological distress and positive affect.

$$\mathbb{E} \left[\left(\text{Stress}_d^{\text{CARE}} - \phi^{\text{CARE}}(S_{t(d)})^\top \theta^{\text{CARE}} - \pi_{t(d)}^{\text{CARE}} - (A_{t(d)}^{\text{CARE}} - \pi_{t(d)}^{\text{CARE}}) \tau^{\text{CARE}} \right) \begin{bmatrix} \phi^{\text{CARE}}(S_{t(d)}) \\ \pi_{t(d)}^{\text{CARE}} \\ A_{t(d)}^{\text{CARE}} - \pi_{t(d)}^{\text{CARE}} \end{bmatrix} \right] = 0.$$

Game intervention onto relational quality.

$$\mathbb{E} \left[\left(\text{Relatsh}_w^{\text{GAME}} - \phi^{\text{GAME}}(S_{t(w)})^\top \theta^{\text{GAME}} - \pi_{t(w)}^{\text{GAME}} - (A_{t(w)}^{\text{GAME}} - \pi_{t(w)}^{\text{GAME}}) \tau^{\text{GAME}} \right) \begin{bmatrix} \phi^{\text{GAME}}(S_{t(w)}) \\ \pi_{t(w)}^{\text{GAME}} \\ A_{t(w)}^{\text{GAME}} - \pi_{t(w)}^{\text{GAME}} \end{bmatrix} \right] = 0.$$

A Trick to Deal with Heterogeneity in Action Space

Our online algorithm is a multi-agent RL algorithm, where each agent makes decisions on a different time-scale. This is due to the heterogeneity in the action space. To apply the approach developed in the existing literature, we need to deal with this heterogeneity.

Our state transition is given by

$$S_{t+1} = \begin{cases} \mathbb{P}_t(\cdot \mid S_t, A_t^{\text{AYA}}, A_t^{\text{CARE}}, A_t^{\text{GAME}}), & \text{if } t \text{ is a new week,} \\ \mathbb{P}_t(\cdot \mid S_t, A_t^{\text{AYA}}, A_t^{\text{CARE}}), & \text{if } t \text{ is a new day,} \\ \mathbb{P}_t(\cdot \mid S_t, A_t^{\text{AYA}}), & \text{otherwise.} \end{cases}$$

The trick we use here is to create dummy action when the daily action or the weekly action is not needed. Specifically, the action space at any t is a three dimensional vector:

- t is a new week w : $\mathbf{A}_t = (A_t^{\text{AYA}}, A_t^{\text{CARE}}, A_t^{\text{GAME}})$.
- t is a new day d : $\mathbf{A}_t = (A_t^{\text{AYA}}, A_t^{\text{CARE}}, A_t^{\text{Dummy}})$.
- otherwise: $\mathbf{A}_t = (A_t^{\text{AYA}}, A_t^{\text{Dummy}}, A_t^{\text{Dummy}})$.

The new transition is given by

$$S_{t+1} = \tilde{\mathbb{P}}_t(\cdot \mid S_t, \mathbf{A}_t),$$

where we write $\mathbf{A}_t = (\mathbf{A}_t^{(1)}, \mathbf{A}_t^{(2)}, \mathbf{A}_t^{(3)})$ and

$$\tilde{\mathbb{P}}_t(\cdot \mid S_t, \mathbf{A}_t) = \begin{cases} \mathbb{P}_t(\cdot \mid S_t, \mathbf{A}_t^{(1)}, \mathbf{A}_t^{(2)}, \mathbf{A}_t^{(3)}), & \text{if } t \text{ is a new week,} \\ \mathbb{P}_t(\cdot \mid S_t, \mathbf{A}_t^{(1)}, \mathbf{A}_t^{(2)}), & \text{if } t \text{ is a new day,} \\ \mathbb{P}_t(\cdot \mid S_t, \mathbf{A}_t^{(1)}), & \text{otherwise.} \end{cases}$$

An important property of Kelly’s approach is that the policy need to converge when $N \rightarrow \infty$.

We first set the dummy action $A_t^{\text{Dummy}} \sim \mathcal{U}\{0, 1\}$. Assume that the care-partner agent and the game agent satisfy that $\pi_{i,t}^{\text{CARE}} = \pi_{i,t}^{\text{CARE}}(\cdot; \beta_{i,t}^{\text{CARE}})$ and $\pi_{i,t}^{\text{GAME}} = \pi_{i,t}^{\text{GAME}}(\cdot; \beta_{i,t}^{\text{GAME}})$, respectively. Then we need

$$\beta_{i,t}^{\text{CARE}} \xrightarrow{P} \beta^{\text{CARE}}, \quad \beta_{i,t}^{\text{GAME}} \xrightarrow{P} \beta^{\text{GAME}} \text{ as } i, N \rightarrow \infty,$$

where β^{CARE} and β^{GAME} are some fixed target policy parameters.

Denote by $\boldsymbol{\theta}_{i,t}$ be the joint vector for $(\beta_{i,t}^{\text{AYA}}, \beta_{i,t}^{\text{CARE}}, \beta_{i,t}^{\text{GAME}})$. Then we have

$$\boldsymbol{\theta}_{i,t} \xrightarrow{P} \boldsymbol{\theta} \text{ as } i, N \rightarrow \infty.$$

where $\boldsymbol{\theta}$ is some fixed target policy parameter.

References

Zhang, K. W., Janson, L., and Murphy, S. A. (2022). Statistical inference after adaptive sampling for longitudinal data. *arXiv preprint arXiv:2202.07098*.